

# CUDA Chapter 1 Notes: Introduction to Heterogeneous Parallel Computing

## 1 Section 1.1: Heterogeneous Parallel Computing

### 1.1 From faster single cores to parallel execution

The central motivation for modern parallel computing is:

**Modern performance improvement comes less from making one instruction stream faster and more from executing many instruction streams in parallel.**

Historically, processor performance improved through higher clock frequencies and increasingly aggressive single-core execution. Power consumption and heat dissipation eventually limited continued frequency scaling, so processor design shifted toward multicore and many-thread architectures. Software must therefore expose parallelism to benefit from modern hardware.

### 1.2 CPU versus GPU design philosophy

CPUs and GPUs follow different optimization goals:

- **CPU: latency-oriented.** A CPU uses a relatively small number of powerful cores, large caches, branch prediction, out-of-order execution, and sophisticated control logic to make a small number of threads finish quickly.
- **GPU: throughput-oriented.** A GPU devotes more hardware resources to arithmetic units and supports a very large number of concurrent threads. Its goal is to maximize total work completed per unit time rather than minimize the latency of one thread.

|                       |                          |
|-----------------------|--------------------------|
| CPU: minimize latency | GPU: maximize throughput |
|-----------------------|--------------------------|

At the architectural level, this difference can be understood in terms of chip-resource allocation: CPUs devote a larger fraction of hardware to control logic and caches, whereas GPUs devote a larger fraction to parallel arithmetic execution.

The GPU is therefore effective when a workload exposes large-scale **data parallelism**, for example vector operations, matrix operations, image processing, physical simulation, and deep-learning workloads.

### 1.3 CUDA and heterogeneous execution

A GPU does not replace the CPU. CUDA(Compute Unified Device Architecture) uses a **heterogeneous computing** model in which the CPU and GPU cooperate:

- the **CPU (host)** handles sequential control, orchestration, input/output, and irregular work;

- the **GPU (device)** handles computationally intensive, massively data-parallel work.

The GPU function executed by many parallel threads is called a **kernel**. A typical CUDA workflow is:

```
// CPU / host
prepare input data;
allocate GPU memory;
copy input data from CPU memory to GPU memory;

// GPU / device
launch a CUDA kernel with many parallel threads;

// CPU / host
copy results from GPU memory back to CPU memory;
postprocess or store the results;
```

CUDA made general-purpose GPU computing practical by allowing programmers to write GPU kernels using a C/C++-like programming model instead of expressing numerical computation through graphics APIs.

## 2 Section 1.2: Why More Speed or Parallelism?

Modern applications continue to demand more computing speed because they process larger datasets, simulate more complex systems, require higher spatial or temporal resolution, and often need results within strict time limits. Representative workloads include scientific simulation, medical computation, image and video processing, physical simulation, and deep learning.

The key computational pattern is **data parallelism**: the same operation is applied independently or nearly independently to many data elements. For example,

$$C_{ij} = A_{ij} + B_{ij} \quad (1)$$

allows different matrix elements to be processed simultaneously by different threads. Large amounts of this type of parallel work are what make GPU acceleration effective.

## 3 Section 1.3: Speeding Up Real Applications

### 3.1 Speedup

The speedup of a new implementation over an old one is

$$S = \frac{T_{\text{old}}}{T_{\text{new}}}. \quad (2)$$

A large GPU peak throughput does not automatically translate into the same application-level speedup. Real performance depends on how much of the program can actually benefit from parallel execution.

### 3.2 Amdahl's Law

Let  $p$  be the fraction of the original execution time that can be parallelized and let  $s$  be the speedup of that parallel portion. The total application speedup is

$$S_{\text{total}} = \frac{1}{(1-p) + \frac{p}{s}}. \quad (3)$$

Even if the parallel portion becomes arbitrarily fast,

$$\lim_{s \rightarrow \infty} S_{\text{total}} = \frac{1}{1 - p}. \quad (4)$$

Therefore, the **sequential fraction sets an upper bound on total speedup**. Optimizing a small parallel region cannot compensate for a large sequential portion of the application.

### 3.3 Sequential and data-parallel portions

Real applications usually contain both types of work:

- **Sequential portions:** initialization, control flow, parameter processing, input/output, and irregular decision logic. These are usually better suited to the CPU.
- **Data-parallel portions:** matrix operations, image processing, neural-network layers, particle updates, and grid-based numerical computation. These are usually better suited to the GPU.

The objective is not to move the entire program to the GPU. The objective is to identify the dominant data-parallel regions and accelerate them while keeping control-oriented work on the CPU.

### 3.4 Memory bandwidth as a performance limit

Peak arithmetic throughput is only one hardware limit. Many kernels are constrained by the rate at which data can be moved to and from memory.

For vector addition,

$$C_i = A_i + B_i, \quad (5)$$

one addition requires two input reads and one output write. The amount of arithmetic per byte transferred is small, so the kernel is often **memory-bound**.

Matrix multiplication,

$$C_{ij} = \sum_k A_{ik} B_{kj}, \quad (6)$$

performs many arithmetic operations and can reuse input data. With sufficient reuse, it can become more **compute-bound**.

This motivates a central CUDA optimization principle:

Reduce expensive memory traffic and increase data reuse so that arithmetic hardware can remain busy.

### 3.5 Division of work between CPU and GPU

GPU execution is not automatically beneficial for every workload. Small problem sizes, insufficient parallelism, irregular memory access, and complex control flow may favor the CPU. A heterogeneous program should therefore assign work according to processor strengths:

|                                  |                                     |
|----------------------------------|-------------------------------------|
| CPU: control and sequential work | GPU: large-scale data-parallel work |
|----------------------------------|-------------------------------------|

## 4 Section 1.4: Challenges in Parallel Programming

Writing parallel code is not simply a matter of replacing a sequential loop with many threads. Several recurring performance issues determine whether parallel execution is actually efficient.

## 4.1 Work efficiency

Some sequential algorithms contain dependencies that require a different algorithmic structure for parallel execution. A parallel algorithm may reduce execution depth while performing more total operations. **Work efficiency** therefore asks whether the extra work introduced by parallelization remains acceptable compared with an efficient sequential algorithm.

## 4.2 Memory behavior and arithmetic intensity

A kernel may be limited by memory movement rather than arithmetic throughput. A useful metric is **arithmetic intensity**:

$$\text{Arithmetic Intensity} = \frac{\text{arithmetic operations}}{\text{bytes transferred from memory}}. \quad (7)$$

Low arithmetic intensity often indicates a memory-bandwidth bottleneck. Later CUDA optimization techniques therefore focus on coalesced access, shared-memory reuse, cache behavior, and reducing unnecessary global-memory traffic.

## 4.3 Load imbalance

If different threads receive very different amounts of work, some threads finish early while others continue running. This **load imbalance** reduces hardware utilization and is especially important for irregular workloads such as graph algorithms.

## 4.4 Synchronization and atomic operations

Threads are not always independent. Cooperative algorithms may require synchronization, while multiple threads updating the same memory location may require atomic operations.

```
__syncthreads();          // synchronize threads in a block
atomicAdd(&sum, value);    // atomic update
```

These mechanisms are necessary for correctness in many algorithms, but they also introduce coordination or serialization overhead.